Artificial Intelligence

AI-2002

Lab 10

Supervised Learning

National University of Computer & Emerging Sciences – NUCES – Karachi

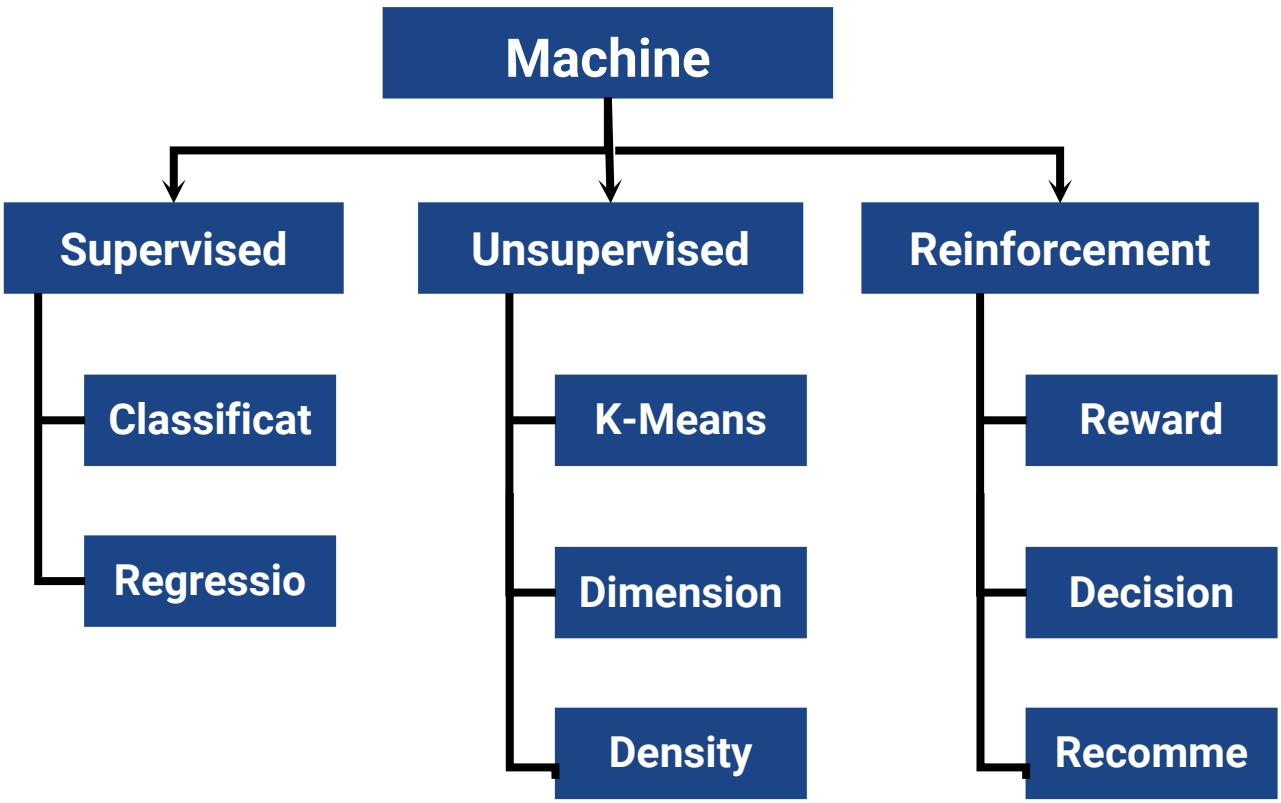
Course Code: AI-2002	Artificial Intelligence Lab

1. Objective

1. The fundamentals of machine learning and its various types.
2. How supervised learning differs from other types of machine learning.
3. The usage of Python libraries like NumPy, Pandas, Matplotlib, and Scikit-learn for implementing machine learning algorithms.
4. Practical implementation of supervised learning classification algorithms such as Linear Regression, Logistic Regression and SVM.
5. Some examples of regression algorithms like Decision Trees.

2. Machine Learning and Its Types

Machine learning is a subset of **artificial intelligence** that enables systems to learn from data and improve their performance without explicit programming. It is broadly categorized into three types:



2.1 Libraries Used in Machine Learning

Below mentioned Python libraries are commonly used to implement machine learning algorithms:

1. **NumPy**: Provides support for numerical computations and working with arrays.
2. **Pandas**: Facilitates data manipulation and analysis.
3. **Matplotlib**: Used for data visualization.
4. **Scikit-learn**: A comprehensive library for implementing both supervised and unsupervised learning algorithms. It also provides tools for data preprocessing, model selection, and evaluation metrics.
5. **Seaborn**: Enhances data visualization by providing advanced plotting functions.

3. Introduction to Supervised Learning

Supervised learning is the type of machine learning in which machines are trained using well **labelled** training data, and on the basis of that data, machines predict the output. The labelled data means some input data is already tagged with the correct output. Supervised learning is a process of providing input data (**features**) as well as correct output data (**labels/targets**) to the machine learning model. The aim of a supervised learning algorithm is to **find a mapping function to map the input variable(x) with the output variable(y).**

3.1 Applications of Supervised Learning

Spam detection
Fraud detection
Predictive maintenance
Sentiment analysis

3.2 Types of Supervised Learning

There are two types of supervised learning algorithms defined as follows:

1. **Classification:** Classification algorithms are used when the output variable is categorical, which means there are two classes such as Yes/No, Male/Female, True/false, etc. Random Forest, Decision Trees, Logistic Regression, Support vector Machines are some of the popular algorithms of classification.
2. **Regression:** Regression algorithms are used if there is a relationship between the input variable and the output variable. It is used for the prediction of continuous variables, such as Weather forecasting, Market Trends, etc. Some popular Regression algorithms which come under supervised learning are Linear Regression, Regression Trees, Non-Linear Regression, Bayesian Linear Regression, Polynomial Regression.

4. Classification

4.1 Support Vector Machine

Support Vector Machines (SVM) are a set of supervised learning algorithms used primarily for classification tasks, but they can also be used for regression. The key idea behind SVM is to find the optimal hyperplane that separates the classes with the largest margin, thus improving generalization. The goal is to maximize the margin between the classes.

Linear SVM: For linearly separable data, SVM finds the hyperplane that perfectly separates the classes. This is the simplest case, and the model does not require any additional tricks.

Non-linear SVM: In many real-world problems, the data may not be linearly separable. In these cases, SVM uses a **kernel function** to map the data into a higher-dimensional feature space, where a linear decision boundary can be found. Popular kernel functions include:

1. **Linear Kernel:** $K(x, x') = x'$
2. **Polynomial Kernel**: $K(x, x') =$
3. **Radial Basis Function (RBF) Kernel**: $K(x, x') =$

The **kernel trick** allows SVM to find a non-linear decision boundary in the original feature space without explicitly calculating the transformation.

```
----------------------------------------------------------------------
-
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target
y = (y == 0).astype(int)  # Convert to binary classification problem

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
```

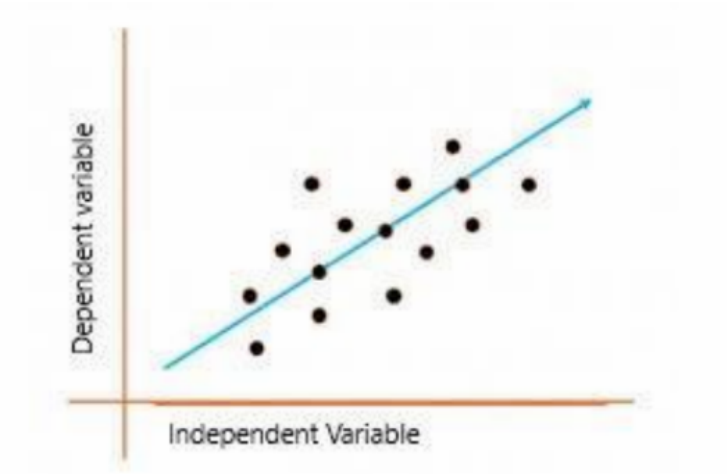
```
random_state=42)

# Train SVM model with RBF kernel
svm = SVC(kernel='rbf', C=1, gamma='scale')
svm.fit(X_train, y_train)

# Make predictions
y_pred = svm.predict(X_test)

# Evaluate the model
print("SVM Accuracy:", accuracy_score(y_test, y_pred))
```

---

5. Regression

5.1 Linear Regression

Linear regression is a quiet and the simplest statistical regression method used for **predictive analysis** in machine learning. Linear regression shows the linear relationship between the independent(predictor) variable i.e. X-axis and the dependent(output) variable i.e. Y-axis, called linear regression. If there is a single input variable X (independent variable), such linear regression is called simple linear regression.



The graph above presents the linear relationship between the output(y) and predictor (X) variables. The blue line is referred to as the best-fit straight line. Based on the given data points, we attempt to plot a line that fits the points the best.
To calculate best-fit line linear regression uses a traditional slope-intercept form which is given below:

$$Yi = \beta 0 + \beta 1 Xi$$

where Yi = Dependent variable, β0 = constant/Intercept, β1 = Slope/Intercept, Xi = Independent variable. This algorithm explains the linear relationship between the dependent(output) variable y and the independent(predictor) variable X using a straight line.

$$Y= B0 + B1 X.$$

The goal of the linear regression algorithm is to get the best values for B0 and B1 to find the best fit line. The best fit line is a line that has the least error which means the error between predicted values and actual values should be minimum.

5.1.2 HOW TO IMPLEMENT LINEAR REGRESSION?

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Split the data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
random_state=42)

# Create and train the Linear Regression model
LR = LinearRegression()
ModelLR = LR.fit(x_train, y_train)

# Predict on the test data
PredictionLR = ModelLR.predict(x_test)

# Print the predictions
print("Predictions:", PredictionLR)
```

5.1.3 HOW TO CALCULATE ERROR IN REGRESSION?

Evaluation Metrics for Linear Regression The strength of any linear regression model can be assessed using various evaluation metrics. These evaluation metrics usually provide a measure of how well the observed outputs are being generated by the model. The most used metrics are:

1. Coefficient of Determination or R-Squared (R2)

2. Root Mean Squared Error (RSME)

Root Mean Squared Error: The Root Mean Squared Error is the square root of the variance of the residuals. It specifies the absolute fit of the model to the data i.e. how close the observed data points are to the predicted values.

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2.$$

Calculating Testing Accuracy

Random Error (Residuals)

In regression, the difference between the observed value of the dependent variable(yi) and the predicted value(predicted) is called the residuals.

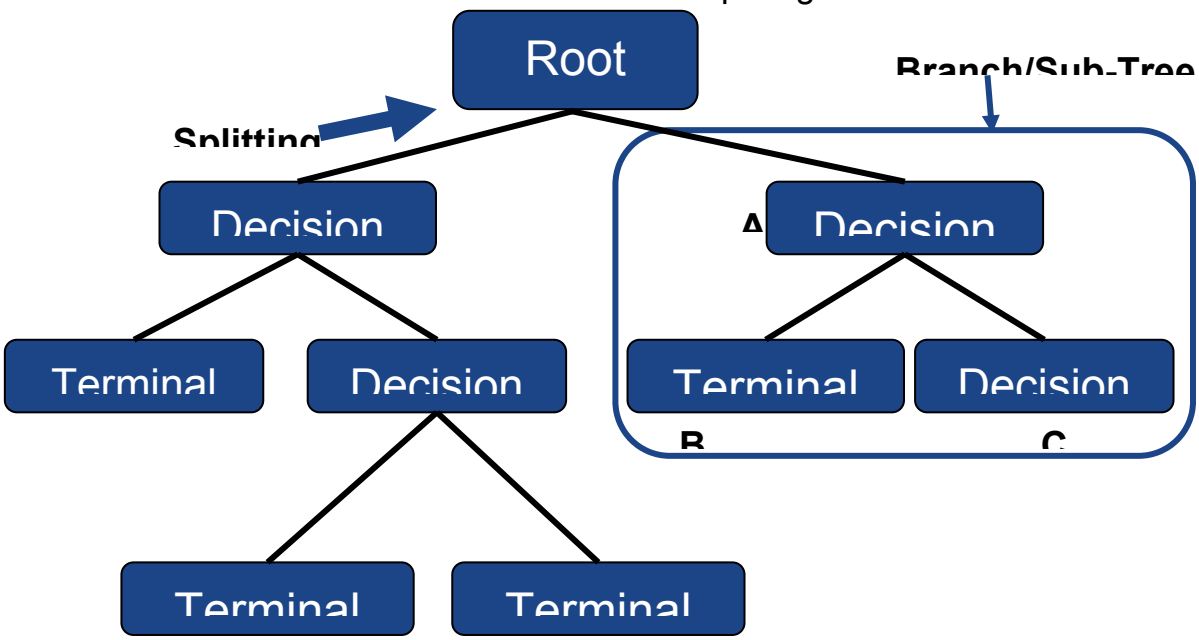
$$\varepsilon i = ypredicted - yi$$

where ypredicted = B0 + B1 Xi

```
from sklearn.metrics import r2_score
print('==================LR Testing Accuracy================')
teachLR = r2_score(y_test, PredictionLR)
testingAccLR = teachLR * 100
print(testingAccLR)
```

5.2 Decision Tree Classifier

Decision tree is the most powerful and popular tool for classification and prediction. A Decision tree is a flowchart like tree structure, where each internal node denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (terminal node) holds a class label. Decision trees can be used for classification as well as regression problems. The name itself suggests that it uses a flowchart like a tree structure to show the predictions that result from a series of feature- based splits. It starts with a root node and ends with a decision made by leaves.



NOTE: A is parent node of B

5.2.1 DECISION TREE TERMINOLOGIES

1. **Root Node:** The first split which decides the entire population or sample data should further get divided into two or more homogeneous sets
2. **Splitting:** It is a process of dividing a node into two or more sub-nodes
3. **Decision Node:** This node decides whether/when a sub-node splits into further sub-nodes or not
4. **Leaf Node:** Terminal Node that predicts the outcome (categorical or continuous value).
5. **Branch:** A subsection of the entire tree is called branch or sub-tree.
6. **Parent Node:** A node divided into sub-nodes is called a parent node of sub-nodes whereas sub-nodes are the child of a parent node.

5.2.2 STEPS TO IMPLEMENT DECISION TREE

Step-1: Begin the tree with the root node, says S, which contains the complete dataset.

Step 2: Find the best attribute in the dataset using Attribute Selection Measure (ASM).

Step 3: Divide the S into subsets that contain possible values for the best attributes.

Step 4: Generate the decision tree node containing the best attribute.

Step-5: Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and call the final node a leaf node.

5.2.3 IMPLEMENTING DECISION TREE

```
from sklearn.tree import DecisionTreeClassifier
# Initialize the DecisionTreeClassifier
DT = DecisionTreeClassifier()

# Train the model
ModelDT = DT.fit(x_train, y_train)

# Model Testing (Prediction)
PredictionDT = DT.predict(x_test)
print("Predictions:", PredictionDT)

# Model Training Accuracy
print('====================DT Training Accuracy===============')
tracDT = DT.score(x_train, y_train)  #  The score method gives accuracy
directly
TrainingAccDT = tracDT * 100
print(f"Training Accuracy: {TrainingAccDT:.2f}%")

# Model Testing Accuracy
print('====================DT Testing Accuracy================')
teacDT = accuracy_score(y_test, PredictionDT)
testingAccDT = teacDT * 100
print(f"Testing Accuracy: {testingAccDT:.2f}%")
```

LAB TASKS

TASK # 1

Imagine you're a data scientist working for a real estate company. Your task is to build a model to predict house prices based on various features such as the number of bedrooms, square footage, and the location of the house. You have access to a dataset with information about houses, including their square footage, number of bedrooms, number of bathrooms, age of the house, and location (neighborhood). The goal is to predict the **price** of the house, which is a continuous variable. Perform the following task:

Clean the dataset and handle any missing values. Encode categorical variables (e.g., neighborhood) as numeric values.
Identify the relevant features that most likely impact the price.
Evaluate the performance of the model using any metrics.
Predict the price of a house given a new set of features.

TASK # 2

You work as a data scientist for an email service provider. Your task is to develop a model that can classify emails as spam or not spam based on their content. You have a labeled dataset with thousands of emails. Each email has features such as frequency of specific words, length of the email, presence of hyperlinks, and sender's address. The task is to classify an email as spam (1) or not spam (0).

Preprocess the dataset by converting text features to numerical features.
Train a model to classify the emails based on their features.
Evaluate the model's performance.
Deploy the model to classify new incoming emails.

TASK # 3

You work for a retail business and your task is to classify customers into two categories: high-value and low-value customers. The classification is based on customer features such as total spending in the last 6 months, age, number of visits, and purchase frequency. You have a dataset with customer information, including spending habits, frequency of visits, and demographics. The goal is to classify customers into high-value (1) and low-value (0) categories.

Clean the dataset and handle any missing or outlier values. Perform feature scaling if necessary.
Divide the dataset into a training set and a testing set.
Find a separating hyperplane for classification.
Find rules that classify customers based on their features.
Evaluate the performance of the model.